Borg IA Cube

Sistema distribuido de CNN con chat multiplataforma y middleware RabbitMQ

Práctica Calificada 3 · CC4P1 Programación Concurrente y Distribuida · UNI 2026-II

Farid (Integrante01) · Hector (Integrante02) · Jorly (Integrante03)

El problema

El usuario envía la foto de una célula por un chat y recibe un diagnóstico de IA.

- **Cubo A, malaria:** parasitada / no infectada
- **Cubo B, sangre:** 8 tipos de células
- Clientes: **Android, escritorio y web**
- CNN escrita desde cero (sin TensorFlow ni PyTorch)

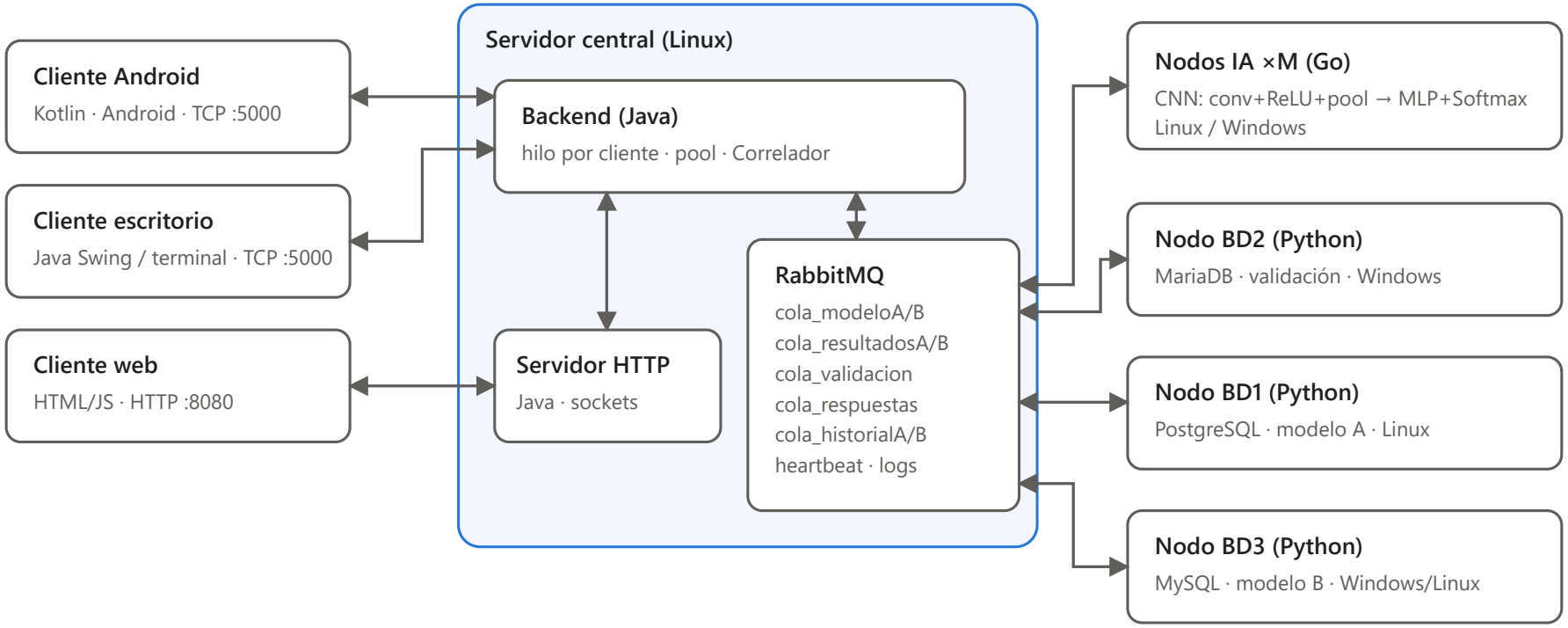
95,2 %

accuracy, modelo A

88,6 %

accuracy, modelo B

Arquitectura

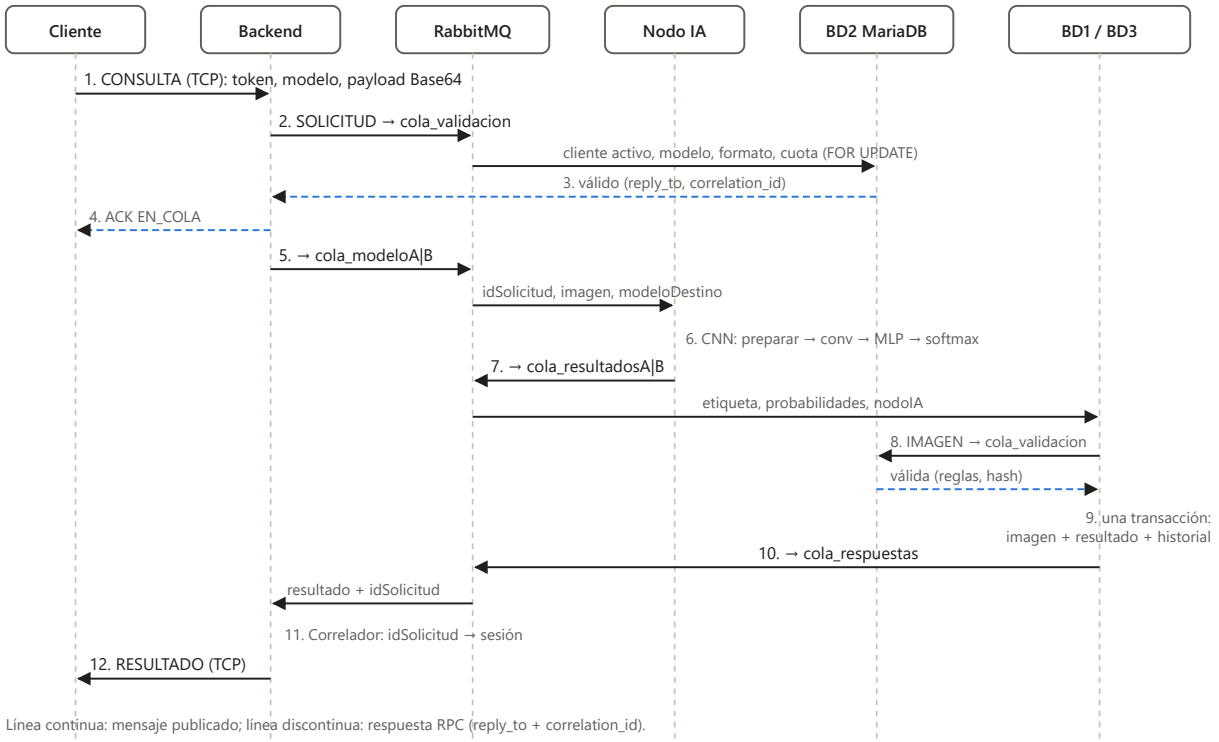


Clientes ↔ backend: sockets TCP propios, trama [4 bytes longitud][JSON] (sin WebSocket).

Backend ↔ nodos IA y BD: solo RabbitMQ (AMQP), mensajes JSON.

3 SO (Linux, Windows, Android) · 3 BD (PostgreSQL, MariaDB, MySQL) · 3 lenguajes (Java, Go, Python) + Kotlin.

Protocolo: viaje de una consulta



- Cliente ↔ backend: **TCP propio**, trama `[4 bytes] [JSON]`
- Backend ↔ nodos: **solo RabbitMQ**
- BD2 valida **2 veces**: cliente/cuota y luego la imagen
- BD1/BD3 guardan en **1 transacción**

El clúster real del equipo

Nodo	SO	Lenguaje	Base
Servidor en Google Cloud (Santiago): RabbitMQ + backend + web	Linux	Java	—
Laptop de Integrante01 (Farid): BD2 + nodo IA	Windows	Python · Go	MariaDB
Laptop de Integrante02: BD1 + nodo IA + carga	Windows	Python · Go	PostgreSQL
Laptop de Integrante03 (Jorly): BD3 + nodo IA	Windows	Python · Go	MySQL
Celular: cliente	Android	Kotlin	—

3 SO · 3 BD en nodos independientes · 4 lenguajes. Todos se conectan hacia la nube: sin abrir puertos en casa.

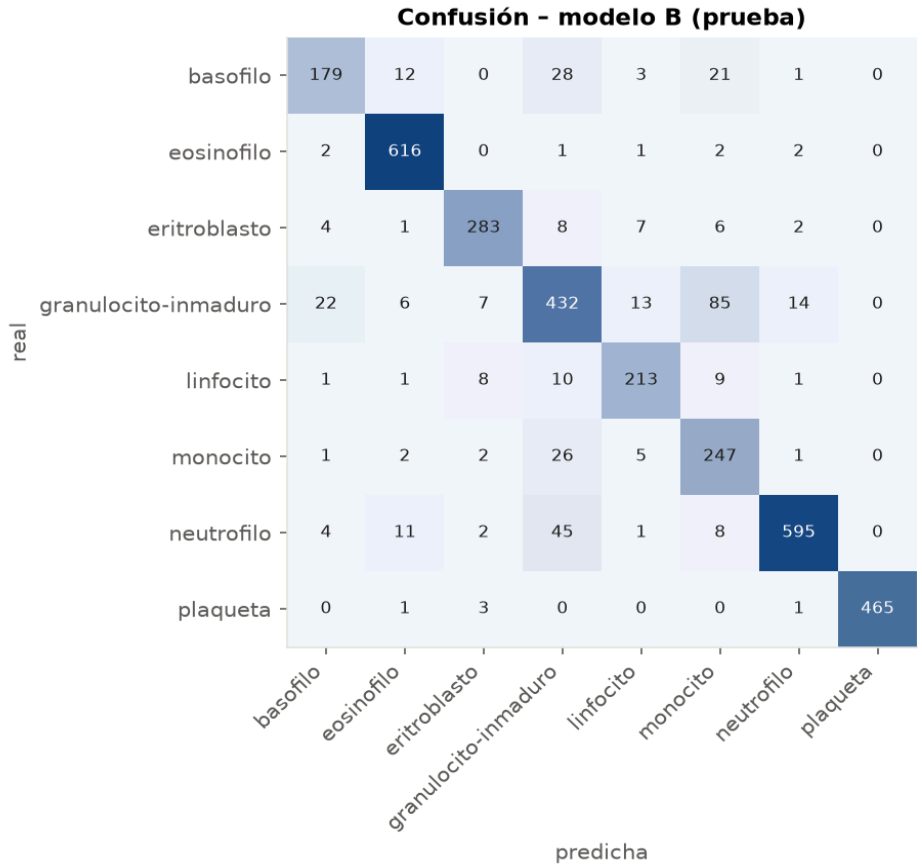
La CNN: dos micro chunks

Convoluciones: conv 3×3 (8) → ReLU → pool → conv 3×3 (16) → ReLU → pool

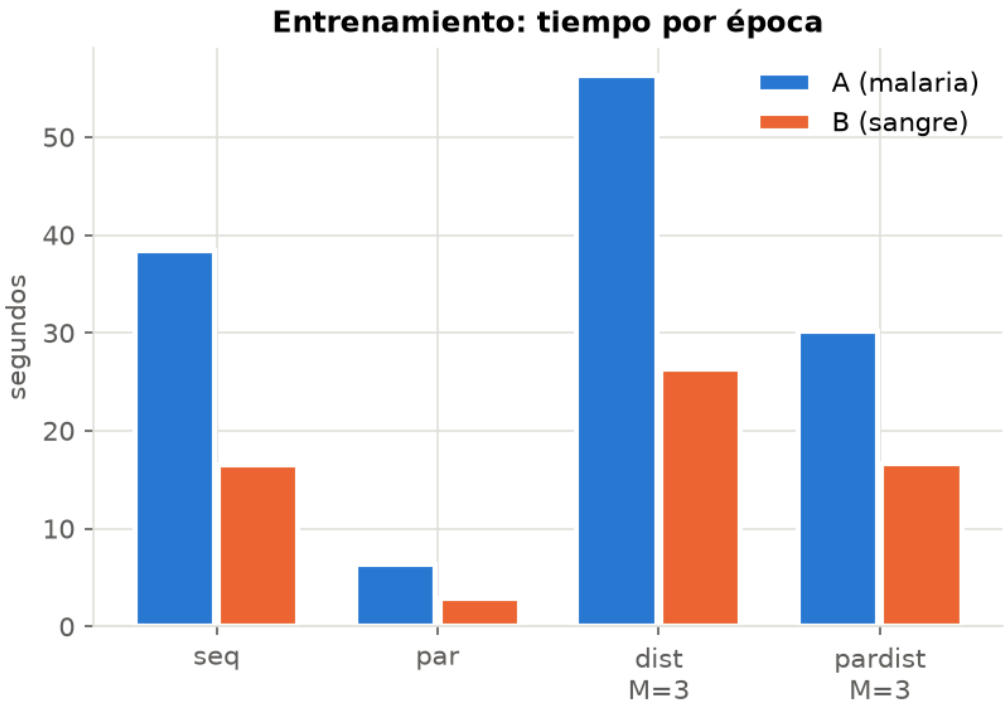
MLP: densa 64 → ReLU → densa K → Softmax

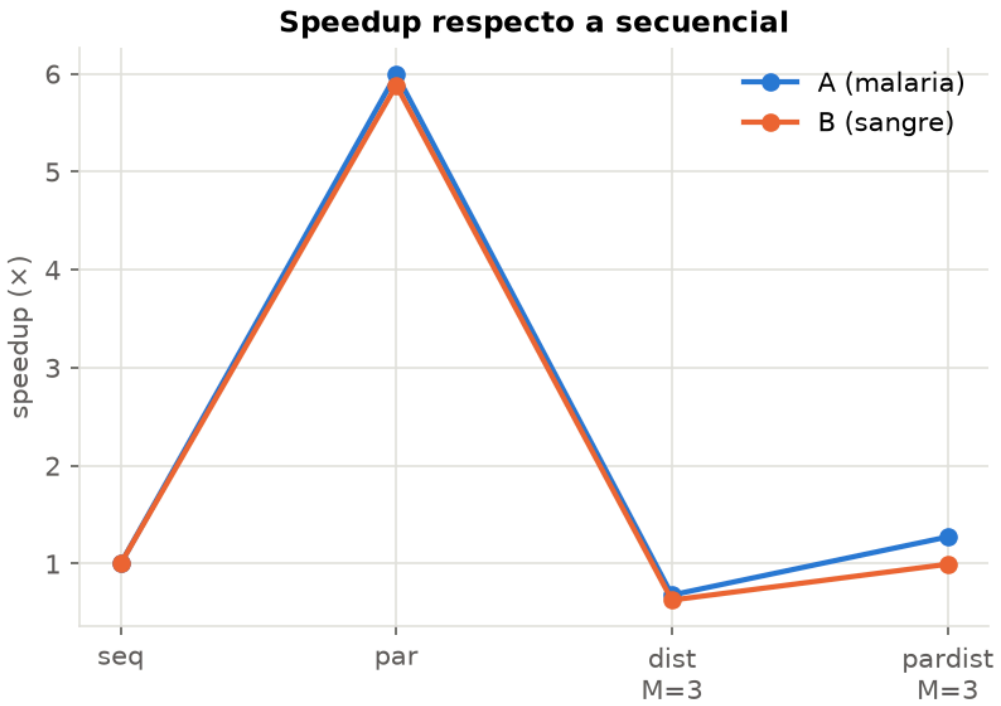
SGD con momentum, backpropagation escrito a mano y verificado con gradiente numérico.

Datasets: NIH Malaria (27 558 imágenes) y BloodMNIST (17 092).



Entrenamiento en 4 configuraciones





Clúster real (3 laptops por internet): paralelo **6,0×** · distribuido **0,7×** · paralelo distribuido **1,3×**

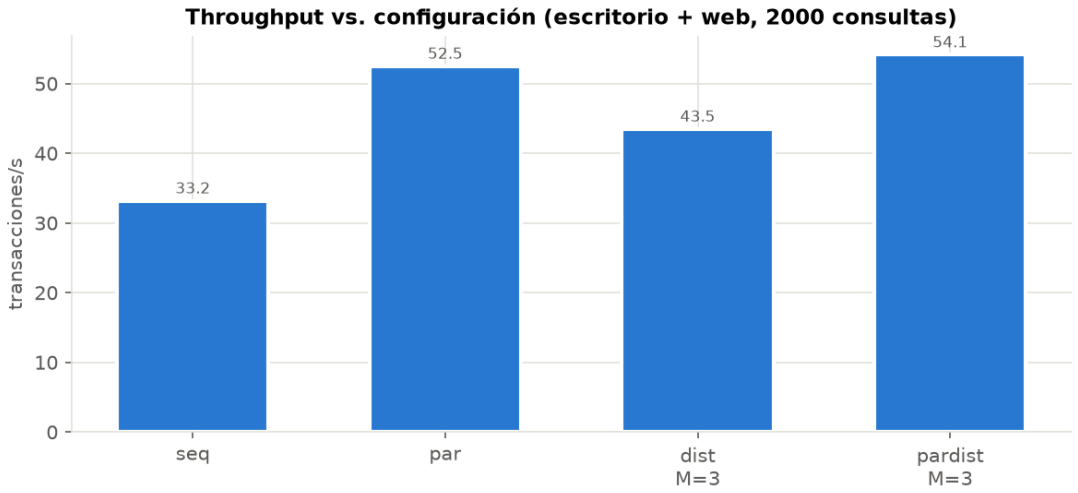
Por internet, enviar el modelo en cada ronda cuesta más que el cálculo (sin red, el distribuido acelera 3,4×): **costo de paralelización**.

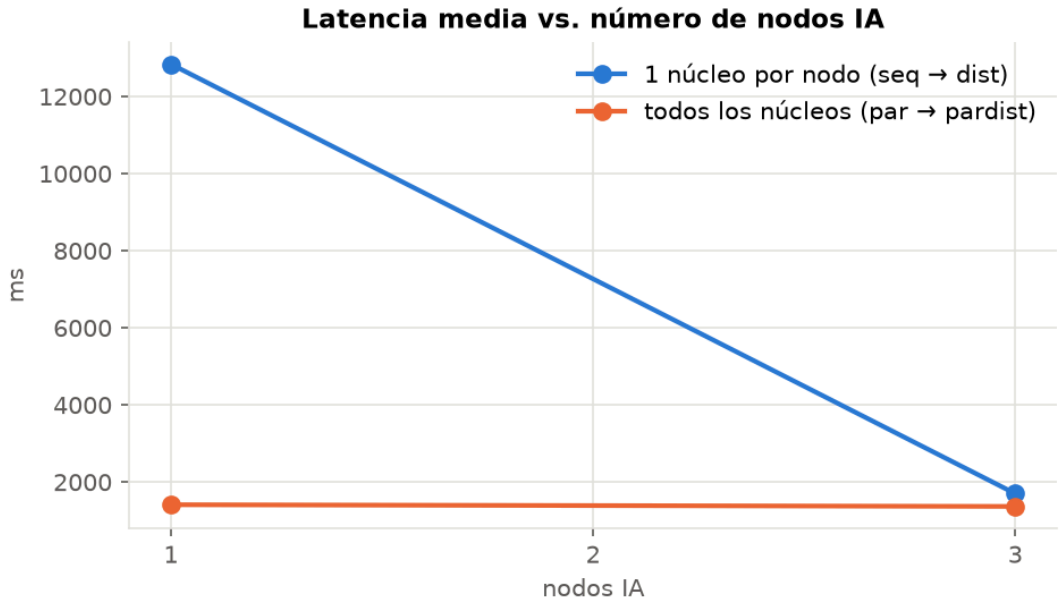
Concurrencia y consistencia

Componente	Hilos	Cómo se evita la condición de carrera
Backend Java	1 por cliente + pool	ReentrantLock por socket, ConcurrentHashMap, remove atómico
Nodo IA Go	H goroutines	Productor-consumidor; un búfer de gradiente por goroutine
Nodos BD Python	Pool de hilos	1 conexión por hilo, transacciones, SELECT … FOR UPDATE, INSERT idempotente
Android Kotlin	Lector + envío	Escritura synchronized, ConcurrentHashMap

Prueba: 50 hilos contra una cuota de 10 → **exactamente 10** aprobadas. 20 hilos con el mismo id → **1 fila**.

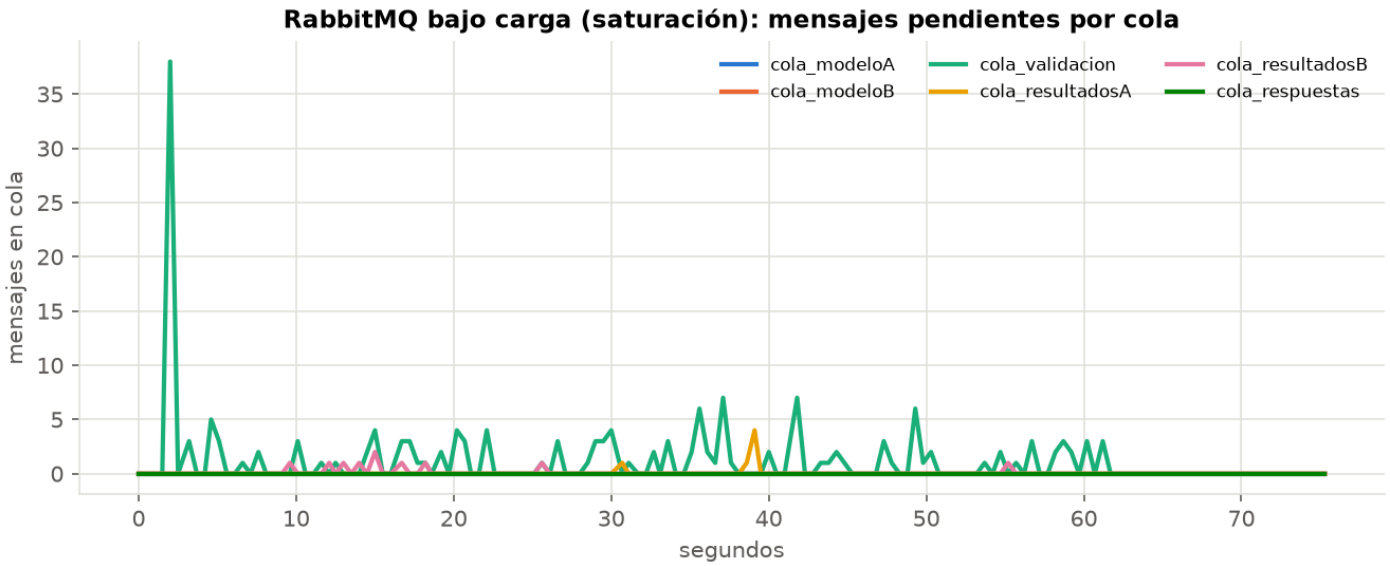
Inferencia: throughput y latencia





Con 1 hilo, la cola de los modelos llega a ~400 mensajes y la latencia a 12,8 s; con más hilos o nodos baja a 1,4 s. Luego el límite pasa a la validación en BD2 (~54 tx/s).

Saturación: Android + escritorio + web a la vez



3000

transacciones

0

errores

Escritorio, web y el celular a la vez, por internet. Ningún nodo pasa del 10 % de CPU: el límite es la red.

Conclusiones

- Flujo completo con los 3 clientes: **sockets propios + RabbitMQ** como único middleware.
- CNN propia: **95,2 %** y **88,6 %**. Entrenamiento paralelo **6×** más rápido; distribuido por internet, más lento por la comunicación.
- RabbitMQ absorbe los picos: **0 errores** en la saturación, y cada cola muestra dónde está el cuello de botella.
- BD1, BD2 y BD3 **consistentes** tras 12 308 consultas y 1000 registros, en 3 casas distintas.

¡Gracias!